

[≡ Blog Menu](#)[≡ Series Contents](#)

50+ Must-know JavaScript Interview Questions by Ex-interviewers (2026)

50+ JavaScript interview questions for 2026, with code examples and answers curated by ex-FAANG interviewers. Covers ES2025—immutable array methods, new Set methods (union/intersection), structuredClone, and modern async patterns.

 [Starter](#) [#javascript](#) [#interviews](#)



GreatFrontEnd Team

50 min read May 20, 2026

[🔗 Copy link](#)[🔗 Share](#)

50+ Must-know JavaScript Interview Questions by Ex-interviewers (2026)

JavaScript is an essential skill for anyone pursuing a career in web development, but securing a job in this field can be particularly challenging for newcomers. A critical part of the hiring process is the technical interview, where your JavaScript expertise will be thoroughly evaluated.

To support your preparation and build your confidence, we've put together a list of the top 75+ must-know JavaScript interview questions and answers frequently encountered in interviews.

"What's new in the May 2026 update"

- *25 new questions on modern JavaScript: optional chaining, nullish coalescing, Promise.allSettled / Promise.any, AbortController, generators, error.cause, immutable array methods (toSorted / toReversed), Object.groupBy, Set union / intersection / difference, iterator helpers, structuredClone, private class fields, and more — see the Modern JavaScript (ES2020+) section below.*

If you're looking for additional JavaScript interview preparation materials, also check out these resources:

- [Front End Interview Playbook](#)
- [Practice JavaScript coding interview questions](#)
- [Practice JavaScript quiz interview questions](#)

1. What is Debouncing in JavaScript?

Debouncing is a smart way to handle events that fire repeatedly within a short time, such as typing in a search box or resizing a window. Instead of executing a function every single time the event is triggered, debouncing ensures the function runs only after the event stops firing for a specified time.

Why is it important?

It prevents performance bottlenecks by reducing the number of unnecessary function calls, making your app smoother and more efficient.

How does it work?

The `debounce` method delays a function's execution until after a defined "waiting period" has passed since the last event. Let's see an example using `Lodash`:

```
1  import { debounce } from 'lodash';
2
3  const searchInput = document.getElementById('search-input');
4
5  const debouncedSearch = debounce(() => {
6    // Perform the search operation here
7    console.log('Searching for:', searchInput.value);
8  }, 300);
9
10 searchInput.addEventListener('input', debouncedSearch);
```

Key features of debouncing

- **Delay-based execution:** Runs the function after user activity has stopped
- **Improves performance:** Prevents excessive computations or network calls during rapid events
- **Flexible configurations:** Supports leading (immediate) and trailing (delayed) execution, and even a maximum wait time

How is it different from throttling?

While **debouncing** waits until user activity stops, **throttling** ensures the function runs at fixed intervals, regardless of how often the event occurs. Each technique suits specific use cases, such as search boxes (debouncing) versus scroll events (throttling).

[Practice implementing a Debounce function on GreatFrontEnd →](#)

2. Understanding `Promise.all()`

`Promise.all()` is a powerful method in JavaScript that allows you to handle multiple asynchronous tasks simultaneously. It takes an array of promises and returns a single promise that resolves when all the promises resolve, or rejects if any one of them fails.

This method is perfect when you need to wait for several independent asynchronous tasks to finish before proceeding, like fetching data from multiple APIs.

Here's how `Promise.all()` works with multiple API requests:

```
1  const promise1 = fetch('https://api.example.com/data/1');
2  const promise2 = fetch('https://api.example.com/data/2');
3  const promise3 = fetch('https://api.example.com/data/3');
4
5  Promise.all([promise1, promise2, promise3])
6    .then((responses) => {
7      // Executes only when all promises are resolved.
```

```
8     console.log('All responses:', responses);
9   })
10  .catch((error) => {
11    // Catches any error from any promise.
12    console.error('Error:', error);
13  });
```

Key Features of `Promise.all`

- **Concurrency:** Runs multiple asynchronous tasks in parallel, improving performance.
- **All-or-nothing resolution:** The promise resolves only when all tasks succeed, or it rejects if any one fails.
- **Simplifies workflows:** Ideal for managing interdependent or independent tasks efficiently.

[Practice implementing a `Promise.all` function on GreatFrontEnd →](#)

3. What is Deep Equal?

Deep equality involves comparing two objects or arrays to determine if they are structurally identical. Unlike shallow equality, which only checks if object references are the same, deep equality examines whether all nested values are equal.

Here's a simple `deepEqual` implementation:

```
1  function deepEqual(obj1, obj2) {
2    if (obj1 === obj2) return true;
3
4    if (
5      obj1 == null ||
6      typeof obj1 !== 'object' ||
7      obj2 == null ||
8      typeof obj2 !== 'object'
9    )
10     return false;
```

```
11
12   let keys1 = Object.keys(obj1);
13   let keys2 = Object.keys(obj2);
14
15   if (keys1.length !== keys2.length) return false;
16
17   for (let key of keys1) {
18     if (!keys2.includes(key) || !deepEqual(obj1[key], obj2[key])) return false;
19   }
20
21   return true;
22 }
23
24 // Example usage
25 const object1 = {
26   name: 'John',
27   age: 30,
28   address: {
29     city: 'New York',
30     zip: '10001',
31   },
32 };
33
34 const object2 = {
35   name: 'John',
36   age: 30,
37   address: {
38     city: 'New York',
39     zip: '10001',
40   },
41 };
42
43 console.log(deepEqual(object1, object2)); // true
```

This function uses recursion to check nested properties, ensuring all values match in both objects or arrays. It's a critical concept for comparing complex data structures in frontend development.

[Practice implementing Deep Equal on GreatFrontEnd →](#)

4. Understanding Event Emitters

An EventEmitter is a utility that enables objects to listen for and emit events. It implements the observer pattern, allowing you to subscribe to actions or changes and handle them when triggered. This concept is fundamental in both JavaScript and Node.js for managing event-driven programming.

```
1  const EventEmitter = new EventEmitter();
2
3  // Subscribe to an event
4  EventEmitter.on('customEvent', (data) => {
5    console.log('Event emitted with data:', data);
6  });
7
8  // Emit the event
9  EventEmitter.emit('customEvent', { message: 'Hello, world!' });
```

EventEmitter allows flexible communication between components, making it useful in scenarios like state management, logging, or real-time updates.

[Practice implementing an Event Emitter on GreatFrontEnd →](#)

5. What is `Array.prototype.reduce()` ?

`Array.prototype.reduce()` is a versatile method for iterating through an array and reducing it to a single value. It processes each element with a callback function, carrying over an accumulator to build the final result. Common use cases include summing numbers, flattening arrays, or even building complex objects.

```
1  const numbers = [1, 2, 3, 4, 5];
2  const sum = numbers.reduce(function (accumulator, currentValue) {
3    return accumulator + currentValue;
4  }, 0);
5
```

```
6 console.log(sum); // Output: 15
```

Why Use reduce ?

- **Flexibility:** Handles various operations, from aggregations to transformations.
- **Functional programming:** Encourages declarative and clean code.
- **Powerful:** Can replace loops or multiple utility methods in a single chain.

[Practice implementing Array.prototype.reduce on GreatFrontEnd →](#)

6. Simplifying arrays – Flattening

Flattening transforms a nested array into a single-level array, making it more manageable. Since ES2019, JavaScript provides the `Array.prototype.flat()` method for this.

```
const nestedArray = [1, [2, [3, [4, [5]]]]];
const flatArray = nestedArray.flat(Infinity);

console.log(flatArray); // Output: [1, 2, 3, 4, 5]
```

Here, `.flat(Infinity)` ensures the entire array is flattened, no matter how deep. For less deeply nested arrays, you can specify the depth.

Before ES2019, custom solutions were common:

```
1 // Custom recursive array flattener
2 function flattenArray(arr) {
3   return arr.reduce(
4     (acc, val) =>
5       Array.isArray(val) ? acc.concat(flattenArray(val)) : acc.concat(val),
6     [],
```

```
7     );  
8   }  
9  
10  const nestedArray = [1, [2, [3, [4, [5]]]]];  
11  const flatArray = flattenArray(nestedArray);  
12  
13  console.log(flatArray); // Output: [1, 2, 3, 4, 5]
```

Practice implementing a flatten function on GreatFrontEnd →

7. Merging data structures

Merging data is crucial when handling complex structures. JavaScript provides efficient ways to combine objects or arrays.

Merging objects

Using the spread operator

The spread operator is concise and intuitive for merging objects:

```
1  const obj1 = { a: 1, b: 2 };  
2  const obj2 = { b: 3, c: 4 };  
3  
4  const mergedObj = { ...obj1, ...obj2 };  
5  
6  console.log(mergedObj); // Output: { a: 1, b: 3, c: 4 }
```

Using Object.assign()

Another approach is `Object.assign()` :

```
1  const obj1 = { a: 1, b: 2 };
```

```
2  const obj2 = { b: 3, c: 4 };
3
4  const mergedObj = Object.assign({}, obj1, obj2);
5
6  console.log(mergedObj); // Output: { a: 1, b: 3, c: 4 }
```

Merging arrays

Using the spread operator

```
1  const array1 = [1, 2, 3];
2  const array2 = [4, 5, 6];
3
4  const mergedArray = [...array1, ...array2];
5
6  console.log(mergedArray); // Output: [1, 2, 3, 4, 5, 6]
```

Using Array.concat()

```
1  const array1 = [1, 2, 3];
2  const array2 = [4, 5, 6];
3
4  const mergedArray = array1.concat(array2);
5
6  console.log(mergedArray); // Output: [1, 2, 3, 4, 5, 6]
```

Deep merging

For nested objects, you'll need custom logic or libraries:

```
1  function deepMerge(target, source) {
2    for (const key in source) {
```

```
3     if (source[key] instanceof Object && key in target) {
4         Object.assign(source[key], deepMerge(target[key], source[key]));
5     }
6 }
7 Object.assign(target || {}, source);
8 return target;
9 }
10
11 const obj1 = { a: 1, b: { x: 10, y: 20 } };
12 const obj2 = { b: { y: 30, z: 40 }, c: 3 };
13
14 const mergedObj = deepMerge(obj1, obj2);
15
16 console.log(mergedObj); // Output: { a: 1, b: { x: 10, y: 30, z: 40 }, c: 3 }
```

Alternatively, libraries like Lodash simplify deep merging:

```
1 const _ = require('lodash');
2
3 const obj1 = { a: 1, b: { x: 10, y: 20 } };
4 const obj2 = { b: { y: 30, z: 40 }, c: 3 };
5
6 const mergedObj = _.merge({}, obj1, obj2);
7
8 console.log(mergedObj); // Output: { a: 1, b: { x: 10, y: 30, z: 40 }, c: 3 }
```

[Practice implementing a deep merge function on GreatFrontEnd →](#)

8. Selecting DOM Elements – `getElementsByClassName`

`getElementsByClassName` fetches elements matching a specific class and returns them as a live `HTMLCollection` .

```
1 // Fetch and loop through elements
2 const elements = document.getElementsByClassName('example');
```

```
3
4  for (let i = 0; i < elements.length; i++) {
5      console.log(elements[i].textContent);
6  }
```

Multiple classes

You can combine class names for more specific selections:

```
const elements = document.getElementsByClassName('class1 class2');
```

Live collections

HTMLCollection updates automatically if DOM elements are added or removed.

For more complex selectors, use `querySelectorAll` :

```
const elements = document.querySelectorAll('.example');
```

[Practice Using `getElementsByClassName` on GreatFrontEnd →](#)

9. Avoiding redundant computations with memoization

Memoization saves computed results to avoid redundant calculations.

```
1  function expensiveOperation(n) {
2      console.log('Calculating for', n);
3      return n * 2;
4  }
5
```

```
6 // Memoize function
7 function memoize(func) {
8   const cache = {};
9
10  return function (n) {
11    if (cache[n] !== undefined) {
12      console.log('From cache for', n);
13      return cache[n];
14    }
15    const result = func(n);
16    cache[n] = result;
17    return result;
18  };
19 }
20
21 const memoizedExpensiveOperation = memoize(expensiveOperation);
22
23 console.log(memoizedExpensiveOperation(5)); // Calculating for 5, 10
24 console.log(memoizedExpensiveOperation(5)); // From cache for 5, 10
```

Libraries like Lodash also provide a `memoize` utility.

[Practice implementing a memoize function on GreatFrontEnd →](#)

10. Safer nested property access: `get`

Accessing nested object properties risk errors if any property is undefined. Tools like Lodash's `get` or JavaScript's optional chaining (`?.`) help mitigate this.

```
const user = { address: { city: 'New York' } };

console.log(_.get(user, 'address.city')); // 'New York'
console.log(user.address?.city); // 'New York'
```

These methods safely retrieve nested properties without crashing the program.

[Practice implementing a `get` function on GreatFrontEnd →](#)

11. Hoisting in JavaScript

Hoisting refers to how JavaScript moves variable and function declarations to the top of their scope during compilation. While only the declaration is hoisted (not the initialization), understanding hoisting helps in writing cleaner and bug-free code.

Hoisting with `var`

Variables declared with `var` are hoisted and initialized as `undefined`. Accessing them before initialization results in `undefined`.

```
console.log(foo); // undefined
var foo = 1;
console.log(foo); // 1
```

Hoisting with `let`, `const`, and `class`

Variables declared with `let`, `const`, and `class` are hoisted but exist in a "temporal dead zone" until their declaration is reached, causing a `ReferenceError` if accessed early.

```
console.log(y); // ReferenceError
let y = 'local';
```

Function hoisting

Function declarations

Both the declaration and definition of functions are hoisted, allowing them to be called before their declaration.

```
1  foo(); // 'FOOOOO'  
2  
3  function foo() {  
4    console.log('FOOOOO');  
5  }
```

Function expressions

For function expressions, only the variable is hoisted, not the function itself.

```
1  console.log(bar); // undefined  
2  bar(); // TypeError: bar is not a function  
3  
4  var bar = function () {  
5    console.log('BARRRR');  
6  };
```

Import statements

Imports are hoisted, making them available throughout the module. However, their initialization happens before the module code executes.

```
foo.doSomething(); // Works fine
```

```
import foo from './modules/foo';
```

Best practices

Modern JavaScript uses `let` and `const` to avoid hoisting pitfalls. Declare variables at the top of their scope for better readability and use tools like ESLint to enforce best practices:

- no-use-before-define
- no-undef

By following these practices, you can write robust, maintainable code.

[Read more about the concept of "Hoisting" on GreatFrontEnd →](#)

12. What are the differences between JavaScript variables created using `let` , `var` , and `const` ?

In JavaScript, `let` , `var` , and `const` are used to declare variables, but they differ in scope, initialization, redeclaration, reassignment, and behavior when accessed before declaration.

Scope

Variables declared with `var` are function-scoped or global, while `let` and `const` are block-scoped (confined to the nearest `{}` block).

```
1  if (true) {  
2    var foo = 1;  
3    let bar = 2;  
4    const baz = 3;  
5  }  
6  
7  console.log(foo); // 1  
8  console.log(bar); // ReferenceError  
9  console.log(baz); // ReferenceError
```

Initialization

`var` and `let` can be declared without initialization, but `const` requires an initial value.

```
var a; // Valid  
let b; // Valid  
const c; // SyntaxError: Missing initializer
```

Redeclaration

Variables declared with `var` can be redeclared, but `let` and `const` cannot.

```
1  var x = 10;
2  var x = 20; // Allowed
3
4  let y = 10;
5  let y = 20; // SyntaxError: Identifier 'y' has already been declared
```

Reassignment

`var` and `let` allow reassignment, while `const` does not.

```
1  let a = 1;
2  a = 2; // Allowed
3
4  const b = 1;
5  b = 2; // TypeError: Assignment to constant variable
```

Access before declaration

All variables are hoisted, but `var` initializes to `undefined`, whereas `let` and `const` exist in a "temporal dead zone" until the declaration is reached.

```
1  console.log(foo); // undefined
2  var foo = 'foo';
3
4  console.log(bar); // ReferenceError
5  let bar = 'bar';
```

Best practices

- Use `const` for variables that don't change to ensure immutability.
- Use `let` when reassignment is needed.
- Avoid `var` due to its hoisting and scoping issues.
- Use tools like ESLint to enforce modern best practices

[Read more about the differences between `let`, `var`, and `const` on GreatFrontEnd →](#)

13. Explain the difference between `==` and `===` in JavaScript?

The `==` operator checks for equality after performing type conversion, while `===` checks for strict equality without type conversion.

Loose equality (`==`)

`==` allows type coercion, which means JavaScript converts values to the same type before comparison. This can lead to unexpected results.

```
42 == '42'; // true
0 == false; // true
null == undefined; // true
```

Strict equality (`===`)

`===` checks both value and type, avoiding the pitfalls of type coercion.

```
42 === '42'; // false
0 === false; // false
null === undefined; // false
```

Use cases

- Prefer `===` for most comparisons as it avoids implicit type conversion and makes code more predictable.
- Use `==` only when comparing `null` or `undefined` for simplicity.
-

```
let x = null;  
console.log(x == null); // true  
console.log(x == undefined); // true
```

Bonus: Object.is()

`Object.is()` is similar to `===` but treats `-0` and `+0` as distinct and considers `NaN` equal to itself.

```
console.log(Object.is(-0, +0)); // false  
console.log(Object.is(NaN, NaN)); // true
```

Conclusion

- Use `===` for strict comparisons to avoid bugs caused by type coercion.
- Rely on `Object.is()` for nuanced comparisons like distinguishing `-0` and `+0`.

[Explore the differences between `==` and `===` on GreatFrontEnd →](#)

14. Understanding the Event Loop in JavaScript

The event loop is the backbone of JavaScript's asynchronous behavior, enabling single-threaded execution without blocking.

Key components

1. **Call stack:** Tracks function executions in a Last-In-First-Out (LIFO) order
2. **Web APIs/Node.js APIs:** Handle asynchronous tasks like `setTimeout` and HTTP requests on separate threads
3. **Task queue (Macrotask queue):** Queues tasks like `setTimeout` and UI events
4. **Microtask queue:** Prioritizes tasks like `Promise` callbacks, executed before macrotasks

How it works

1. **Synchronous code execution:** Functions are pushed and popped from the call stack.
2. **Asynchronous tasks:** Offloaded to APIs for processing.
3. **Task completion:** Completed tasks are queued.
4. **Event loop execution:** Executes microtasks until the queue is empty. Processes one macrotask and checks the microtask queue again.

```
1 console.log('Start');
2
3 setTimeout(() => console.log('Timeout 1'), 0);
4
5 Promise.resolve().then(() => console.log('Promise 1'));
6
7 setTimeout(() => console.log('Timeout 2'), 0);
8
9 console.log('End');
```

Output:

```
1 Start
2 End
3 Promise 1
4 Timeout 1
```

5 Timeout 2

Explanation:

- Synchronous logs (Start , End) run first.
- Microtasks (Promise 1) follow.
- Macrotasks (Timeout 1 , Timeout 2) run last.

[Explore the event loop in JavaScript on GreatFrontEnd →](#)

15. What is Event Delegation in JavaScript?

Event delegation is an efficient way to manage events for multiple elements by attaching a single event listener to their common parent.

How it works

1. **Attach a listener:** Add an event listener to a parent element instead of each child.
2. **Event bubbling:** Events triggered on children bubble up to the parent.
3. **Identify target:** Use `event.target` to determine the clicked element.
4. **Perform action:** Execute logic based on the event target.

```
1  // HTML:
2  // <ul id="item-list">
3  //   <li>Item 1</li>
4  //   <li>Item 2</li>
5  // </ul>
6
7  const itemList = document.getElementById('item-list');
8
9  itemList.addEventListener('click', (event) => {
10    if (event.target.tagName === 'LI') {
11      console.log(`Clicked on ${event.target.textContent}`);
```

```
12     }  
13   });
```

Benefits

1. **Efficiency:** Reduces the number of event listeners, improving performance.
2. **Dynamic content:** Automatically handles new elements added to the DOM.

[Explore event delegation in JavaScript on GreatFrontEnd →](#)

16. How `this` works in JavaScript

The value of `this` depends on how a function is called. Let's explore its different behaviors.

Scenarios

1. Using `new` : When creating objects, `this` refers to the newly created object.

```
1  function Person(name) {  
2    this.name = name;  
3  }  
4  const person = new Person('Alice');  
5  console.log(person.name); // 'Alice'
```

2. Using `apply` , `call` , or `bind` : Explicitly sets `this` to a specified object.

```
1  function greet() {  
2    console.log(this.name);  
3  }  
4  const person = { name: 'Alice' };  
5  greet.call(person); // 'Alice'
```

3. **Method call:** `this` refers to the object the method is called on.

```
1  const obj = {  
2    name: 'Alice',  
3    greet() {  
4      console.log(this.name);  
5    },  
6  };  
7  obj.greet(); // 'Alice'
```

4. **Free function call:** Defaults to the global object (`window` in browsers) or `undefined` in strict mode.

```
function greet() {  
  console.log(this); // global object or undefined  
}  
greet();
```

5. **Arrow functions:** Capture `this` from their enclosing scope.

```
1  const obj = {  
2    name: 'Alice',  
3    greet: () => {  
4      console.log(this.name); // Inherits `this` from enclosing scope  
5    },  
6  };  
7  obj.greet(); // undefined
```

ES6 and `this`

Arrow functions simplify usage by capturing `this` from their lexical scope.

```
1  function Timer() {  
2    this.seconds = 0;  
3    setInterval(() => {  
4      this.seconds++;  
5      console.log(this.seconds);  
6    }, 1000);  
7  }  
8  
9  const timer = new Timer();
```

[Explore how this works in JavaScript on GreatFrontEnd →](#)

17. What sets Cookies, sessionStorage, and localStorage apart?

When it comes to client-side storage, cookies, localStorage, and sessionStorage serve distinct roles:

Cookies

- **Function:** Stores small pieces of data sent along with HTTP requests to the server.
- **Limit:** Roughly 4KB per domain.
- **Lifetime:** Can persist or expire after a set time. Session cookies disappear when the browser closes.
- **Scope:** Accessible across pages and subdomains for a single domain.
- **Security:** Features like HttpOnly and Secure flags add extra security.

```
1  // Set a cookie with an expiry date  
2  document.cookie = 'userId=12345; expires=Fri, 31 Dec 2025 23:59:59 GMT; path=/';  
3  
4  // Read all cookies  
5  console.log(document.cookie);
```

```
6
7 // Delete a cookie
8 document.cookie = 'userId=; expires=Thu, 01 Jan 1970 00:00:00 GMT; path=/';
```

localStorage

- **Function:** Allows persistent data storage on the client side.
- **Limit:** About 5MB per origin.
- **Lifetime:** Data stays until explicitly removed.
- **Scope:** Shared across all tabs and windows for the same origin.
- **Security:** Accessible by JavaScript within the same origin.

```
1 // Store data in localStorage
2 localStorage.setItem('username', 'john_doe');
3
4 // Retrieve data
5 console.log(localStorage.getItem('username'));
6
7 // Remove an item
8 localStorage.removeItem('username');
9
10 // Clear all localStorage data
11 localStorage.clear();
```

sessionStorage

- **Function:** Stores data for the duration of a page session.
- **Limit:** Similar to localStorage (around 5MB).
- **Lifetime:** Cleared when the tab or browser closes.
- **Scope:** Data is confined to the current tab or window.
- **Security:** Accessible by JavaScript on the same origin.

```
1 // Store data in sessionStorage
2 sessionStorage.setItem('sessionId', 'abcdef');
3
4 // Retrieve data
5 console.log(sessionStorage.getItem('sessionId'));
6
7 // Remove an item
8 sessionStorage.removeItem('sessionId');
9
10 // Clear all sessionStorage data
11 sessionStorage.clear();
```

[Learn more about cookies, sessionStorage, and localStorage on GreatFrontEnd →](#)

18. How do `<script>`, `<script async>`, and `<script defer>` differ?

`<script>`

When using the `<script>` tag without attributes, it fetches and executes the script immediately, pausing HTML parsing.

Use case: Critical scripts needed before page rendering.

```
<script src="main.js"></script>
```

`<script async>`

With `async`, the script loads in parallel to HTML parsing and executes as soon as it's ready.

Use case: Independent scripts like analytics or ads.

```
<script async src="analytics.js"></script>
```

```
<script defer>
```

When using `defer`, the script loads alongside HTML parsing but only executes after the HTML is fully parsed.

Use Case: Scripts that rely on a complete DOM structure.

```
<script defer src="deferred.js"></script>
```

Discover more about `<script>`, `<script async>`, and `<script defer>` on GreatFrontEnd →

19. What's the difference between `null`, `undefined` ?

Undeclared

Variables not defined using `var`, `let`, or `const` are considered undeclared and can cause global scope issues.

undefined

A declared variable that hasn't been assigned a value is `undefined`.

`null`

Represents the intentional absence of any value. It's an explicit assignment. **Example Code:**

```
1  let a;
2  console.log(a); // undefined
3
4  let b = null;
5  console.log(b); // null
6
7  try {
8    console.log(c); // ReferenceError: c is not defined
9  } catch (e) {
10   console.log('c is undeclared');
11 }
```

[Read more about `null` , `undefined` , and undeclared variables on GreatFrontEnd →](#)

20. What's the difference between `.call()` vs `.apply()` ?

Both `.call` and `.apply` let you invoke a function with a specified `this` value. The key difference lies in how arguments are passed:

- `.call` : Accepts arguments as a comma-separated list.
- `.apply` : Accepts arguments as an array.

Memory aid:

- C for call = comma-separated
- A for apply = array

```
1  function sum(a, b) {
2    return a + b;
3  }
4
5  console.log(sum.call(null, 1, 2)); // 3
6  console.log(sum.apply(null, [1, 2])); // 3
```

[Learn more about .call and .apply on GreatFrontEnd →](#)

21. How does `Function.prototype.bind` work?

The `bind` method is used to create a new function with a specific `this` value and, optionally, preset arguments. This ensures that the function always has the correct `this` context, regardless of how or where it's called.

Key uses of `bind` :

1. **Maintaining Context:** Ensures that `this` is correctly set for the function.
2. **Preset Arguments:** Allows you to predefine arguments for a function.
3. **Borrowing Methods:** Enables you to use methods from one object in another.

```
1  const john = {
2    age: 42,
3    getAge: function () {
4      return this.age;
5    },
6  };
7
8  console.log(john.getAge()); // 42
9
10 const unboundGetAge = john.getAge;
11 console.log(unboundGetAge()); // undefined
12
13 const boundGetAge = john.getAge.bind(john);
14 console.log(boundGetAge()); // 42
15
16 const mary = { age: 21 };
17 const boundGetAgeMary = john.getAge.bind(mary);
18 console.log(boundGetAgeMary()); // 21
```

[Explore `Function.prototype.bind` on GreatFrontEnd →](#)

22. Why use arrow functions in constructors?

Using arrow functions for methods in constructors automatically binds the `this` context to the constructor, avoiding the need to manually bind it. This eliminates issues caused by `this` referring to unexpected contexts.

```
1  const Person = function (name) {  
2    this.name = name;  
3    this.sayName1 = function () {  
4      console.log(this.name);  
5    };  
6    this.sayName2 = () => {  
7      console.log(this.name);  
8    };  
9  };  
10  
11 const john = new Person('John');  
12 const dave = new Person('Dave');  
13  
14 john.sayName1(); // John  
15 john.sayName2(); // John  
16  
17 john.sayName1.call(dave); // Dave  
18 john.sayName2.call(dave); // John
```

Arrow functions are particularly useful in React class components, ensuring methods maintain the correct context when passed to child components.

[Explore the advantage for using the arrow syntax for a method in a constructor on GreatFrontEnd →](#)

23. How does prototypal inheritance work?

Prototypal inheritance is a way for objects to share properties and methods through their prototype chain.

Key concepts:

1. **Prototypes:** Each object has a prototype, from which it inherits properties and methods.
2. **Prototype chain:** JavaScript looks for properties/methods up the chain until it finds them or reaches `null`.
3. **Constructor functions:** Functions used with `new` to create objects.

```
1  function Animal(name) {  
2    this.name = name;  
3  }  
4  
5  Animal.prototype.sayName = function () {  
6    console.log(`My name is ${this.name}`);  
7  };  
8  
9  function Dog(name, breed) {  
10   Animal.call(this, name);  
11   this.breed = breed;  
12 }  
13  
14 Dog.prototype = Object.create(Animal.prototype);  
15 Dog.prototype.bark = function () {  
16   console.log('Woof!');  
17 };  
18  
19 let fido = new Dog('Fido', 'Labrador');  
20 fido.bark(); // "Woof!"  
21 fido.sayName(); // "My name is Fido"
```

[Explore how prototypal inheritance works on GreatFrontEnd →](#)

24. Differences between: `function Person(){} , const person = Person() , and const person = new Person() ?`

Key differences:

1. `function Person(){} : A function declaration, typically used for constructors if written in PascalCase.`
2. `const person = Person() : Calls the function normally and assigns the result to person . No object creation happens unless explicitly returned.`
3. `const person = new Person() : Invokes the function as a constructor, creating a new object and setting its prototype to Person.prototype .`

Explore the difference between: `function Person(){} , const person = Person() , and const person = new Person()` on GreatFrontEnd →

25. Function declarations vs. Function expressions

Function declarations:

- **Syntax:** `function foo() {}`
- **Hoisting:** Fully hoisted; can be called before its definition.

```
foo(); // "Hello!"  
function foo() {  
  console.log('Hello!');  
}
```

Function expressions:

- **Syntax:** `var foo = function() {}`
- **Hoisting:** Only the variable is hoisted, not the function body.

```
foo(); // TypeError: foo is not a function
var foo = function () {
  console.log('Hello!');
};
```

Explore the differences on the usage of `foo` between `function foo() {}` and `var foo = function() {}` on GreatFrontEnd →

26. What are the different ways to create objects in JavaScript?

Here are various approaches to creating objects in JavaScript:

1. **Object literals:** The simplest and most common way to create an object is using curly braces `{}` with key-value pairs.

```
const person = {
  firstName: 'John',
  lastName: 'Doe',
};
```

2. **Object constructor:** Use the built-in `Object` constructor with the `new` keyword.

```
const person = new Object();
person.firstName = 'John';
person.lastName = 'Doe';
```

3. **`Object.create()` method:** Create an object with a specific prototype.

```
1  const personPrototype = {
2    greet() {
```

```
3     console.log(`Hello, my name is ${this.name}.`);
4   },
5   };
6
7   const person = Object.create(personPrototype);
8   person.name = 'John';
9   person.greet(); // Hello, my name is John.
```

4. ES2015 classes: Define objects using the `class` syntax for a blueprint-like structure.

```
1   class Person {
2     constructor(name, age) {
3       this.name = name;
4       this.age = age;
5     }
6
7     greet() {
8       console.log(`Hi, I'm ${this.name} and I'm ${this.age} years old.`);
9     }
10  }
11
12  const john = new Person('John', 30);
13  john.greet(); // Hi, I'm John and I'm 30 years old.
```

5. Constructor functions: Use a function as a template for creating multiple objects.

```
1   function Person(name, age) {
2     this.name = name;
3     this.age = age;
4   }
5
6   const john = new Person('John', 30);
7   console.log(john.name); // John
```

[Explore various ways to create objects in JavaScript on GreatFrontEnd →](https://www.greatfrontend.com/blog/50-must-know-javascript-interview-questions-by-ex-interviewers)

27. What is a higher-order function?

A higher-order function is a function that either:

1. Accepts another function as an argument:

```
1  function greet(name) {  
2    return `Hello, ${name}!`;  
3  }  
4  
5  function greetUser(greeter, name) {  
6    console.log(greeter(name));  
7  }  
8  
9  greetUser(greet, 'Alice'); // Hello, Alice!
```

2. Returns another function:

```
1  function multiplier(factor) {  
2    return function (num) {  
3      return num * factor;  
4    };  
5  }  
6  
7  const double = multiplier(2);  
8  console.log(double(4)); // 8
```

[Explore the definition of a higher-order function on GreatFrontEnd →](#)

28. How do ES2015 classes differ from ES5 constructor functions?

ES5 constructor functions use function constructors and prototypes for object creation and inheritance.

```
1  function Person(name, age) {  
2    this.name = name;  
3    this.age = age;  
4  }  
5  
6  Person.prototype.greet = function () {  
7    console.log(`Hi, I'm ${this.name} and I'm ${this.age} years old.`);  
8  };  
9  
10 const john = new Person('John', 30);  
11 john.greet(); // Hi, I'm John and I'm 30 years old.
```

ES2015 Classes use the `class` keyword for cleaner and more intuitive syntax.

```
1  class Person {  
2    constructor(name, age) {  
3      this.name = name;  
4      this.age = age;  
5    }  
6  
7    greet() {  
8      console.log(`Hi, I'm ${this.name} and I'm ${this.age} years old.`);  
9    }  
10 }  
11  
12 const john = new Person('John', 30);  
13 john.greet(); // Hi, I'm John and I'm 30 years old.
```

Key differences:

- **Syntax:** ES2015 classes are more readable and concise.
- **Static methods:** Easier to define using `static` in ES2015.
- **Inheritance:** Simpler with the `extends` and `super` keywords in ES2015.

[Explore differences between ES2015 classes and ES5 constructor functions on GreatFrontEnd →](#)

29. What is event bubbling?

Event bubbling is the process where an event triggers on the target element and then propagates upwards through its ancestors in the DOM.

```
1  const parent = document.getElementById('parent');
2  const child = document.getElementById('child');
3
4  parent.addEventListener('click', () => {
5    console.log('Parent clicked');
6  });
7
8  child.addEventListener('click', () => {
9    console.log('Child clicked');
10 });
```

Clicking the child element will log both "Child clicked" and "Parent clicked" due to bubbling.

Prevent bubbling:

Use `event.stopPropagation()` to prevent the event from propagating upwards.

```
child.addEventListener('click', (event) => {
  event.stopPropagation();
  console.log('Child clicked only');
});
```

[Explore event bubbling on GreatFrontEnd →](#)

30. What is event capturing?

Event capturing, also called "trickling", is the reverse of bubbling. The event propagates from the root element down to the target element.

Enable event capturing

Capturing is enabled by passing `{ capture: true }` to `addEventListener()` as the third argument.

```
1  const parent = document.getElementById('parent');
2  const child = document.getElementById('child');
3
4  parent.addEventListener(
5    'click',
6    () => {
7      console.log('Parent capturing');
8    },
9    { capture: true },
10 );
11
12 child.addEventListener('click', () => {
13   console.log('Child clicked');
14 });
```

Clicking the child will log "Parent capturing" first, followed by "Child clicked."

[Explore event capturing on GreatFrontEnd →](#)

31. How do `mouseenter` and `mouseover` differ?

`mouseenter`

- Does **not** bubble up the DOM tree.

- Triggered only when the mouse pointer enters the element itself, excluding its children.
- Fires a single event when entering the target element.

mouseover

- Bubbles up the DOM tree.
- Triggered when the mouse pointer enters the target element or any of its children.
- Fires multiple events when moving over child elements.

[Explore the differences between mouseenter and mouseover on GreatFrontEnd →](#)

32. What's the difference between synchronous and asynchronous functions?

Synchronous functions

- Execute tasks in a sequential, blocking manner.
- Program execution halts until the current task completes.
- Easier to debug due to their predictable flow.

```
const fs = require('fs');  
const data = fs.readFileSync('file.txt', 'utf8');  
console.log(data); // Blocks until the file is fully read  
console.log('Program ends');
```

Asynchronous functions

- Perform tasks without blocking program execution.
- Other operations can run while waiting for the task to finish.

- Commonly used for I/O operations, network requests, and timers.

```
1 console.log('Start');
2
3 fetch('https://api.example.com/data')
4   .then((response) => response.json())
5   .then((data) => console.log(data)) // Non-blocking
6   .catch((error) => console.error(error));
7
8 console.log('End');
```

Explore the difference between synchronous and asynchronous functions on GreatFrontEnd -

>
=

33. What is AJAX?

AJAX (Asynchronous JavaScript and XML) is a technique that allows web pages to fetch and send data asynchronously, enabling dynamic updates without reloading the entire page.

Key points

- **Asynchronous:** Updates parts of a page without reloading.
- **Data formats:** Initially XML, now primarily JSON due to its simplicity.
- **APIs:** Traditionally used XMLHttpRequest ; fetch() is the modern alternative.

Using XMLHttpRequest :

```
1 let xhr = new XMLHttpRequest();
2 xhr.onreadystatechange = function () {
3   if (xhr.readyState === XMLHttpRequest.DONE) {
4     if (xhr.status === 200) {
5       console.log(xhr.responseText);
```

```
6      } else {  
7          console.error('Request failed');  
8      }  
9  }  
10 };  
11 xhr.open('GET', 'https://jsonplaceholder.typicode.com/todos/1', true);  
12 xhr.send();
```

Using fetch() :

```
fetch('https://jsonplaceholder.typicode.com/todos/1')  
  .then((response) => response.json())  
  .then((data) => console.log(data))  
  .catch((error) => console.error('Fetch error:', error));
```

[Explore AJAX in detail on GreatFrontEnd →](#)

34. What are the pros and cons of using AJAX?

Advantages

- Enhances user experience by enabling seamless updates.
- Reduces server load by fetching only necessary data.
- Keeps the user on the same page while updating content.

Disadvantages

- Relies on JavaScript, so functionality may break if it's disabled.
- SEO challenges with dynamically loaded content.
- Bookmarking specific page states becomes difficult.

Explore the advantages and disadvantages of using AJAX on GreatFrontEnd →

35. What are the differences between XMLHttpRequest and fetch() ?

XMLHttpRequest

- **Syntax:** Event-driven; requires listeners for response handling.
- **Progress tracking:** Supports progress tracking via `onprogress`.
- **Error handling:** Uses `onerror` event.

```
1  let xhr = new XMLHttpRequest();
2  xhr.open('GET', 'https://example.com/api', true);
3  xhr.onload = function () {
4    if (xhr.status === 200) {
5      console.log(xhr.responseText);
6    }
7  };
8  xhr.send();
```

fetch()

- **Syntax:** Promise-based; simpler and more readable.
- **Error handling:** Uses `.catch()` for better error management.
- **Modern features:** Built-in support for `AbortController` for cancellations.

```
fetch('https://example.com/api')
  .then((response) => response.json())
  .then((data) => console.log(data))
```

```
.catch((error) => console.error(error));
```

Key differences

- `fetch()` has cleaner syntax and better Promise integration.
- `XMLHttpRequest` supports progress tracking, which `fetch()` does not.

[Explore the differences between XMLHttpRequest and fetch\(\) on GreatFrontEnd →](#)

36. What are the various data types in JavaScript?

JavaScript features a mix of primitive and non-primitive (reference) data types.

Primitive data types

- **Number:** Includes integers and floating-point values.
- **String:** Text values enclosed in single, double quotes, or backticks.
- **Boolean:** Represents `true` or `false`.
- **Undefined:** A declared variable that hasn't been assigned a value.
- **Null:** Indicates an intentional lack of value.
- **Symbol:** A unique and immutable identifier often used as object property keys.
- **BigInt:** Handles large integers with arbitrary precision.

Non-primitive data types

- **Object:** Collections of key-value pairs.
- **Array:** Ordered lists of elements.
- **Function:** First-class objects that can be assigned, passed, and returned.

- **Date:** Represents date and time values.
- **RegExp:** For pattern matching in strings.
- **Map:** A collection of key-value pairs, allowing any type of key.
- **Set:** Stores unique values, whether primitive or object references.

Tip: Use the `typeof` operator to determine the type of a variable.

[Explore the various data types in JavaScript on GreatFrontEnd →](#)

37. How do you iterate over object properties and array items?

JavaScript provides multiple ways to iterate over objects and arrays.

Iterating over objects

1. `for...in`

Loops over all enumerable properties, including inherited ones.

```
1  for (const property in obj) {  
2    if (Object.hasOwnProperty(obj, property)) {  
3      console.log(property);  
4    }  
5  }
```

2. `Object.keys()`

Retrieves an array of an object's own enumerable properties.

```
Object.keys(obj).forEach((key) => console.log(key));
```

3. Object.entries()

Returns an array of [key, value] pairs.

```
Object.entries(obj).forEach(([key, value]) => console.log(`${key}: ${value}`));
```

4. Object.getOwnPropertyNames()

Includes both enumerable and non-enumerable properties.

```
Object.getOwnPropertyNames(obj).forEach((prop) => console.log(prop));
```

Iterating over arrays

1. for Loop

Classic approach for iterating through arrays:

```
for (let i = 0; i < arr.length; i++) {  
  console.log(arr[i]);  
}
```

2. Array.prototype.forEach()

Executes a callback for each array item.

```
arr.forEach((element, index) => console.log(element, index));
```

3. for...of

Ideal for looping through iterable objects like arrays.

```
for (const element of arr) {  
  console.log(element);  
}
```

4. Array.prototype.entries()

Iterates with both index and value.

```
for (const [index, element] of arr.entries()) {  
  console.log(index, ':', element);  
}
```

[Explore iteration techniques on GreatFrontEnd →](#)

38. What are the benefits of spread syntax, and how is it different from rest syntax?

Spread syntax (...)

The spread operator is used to expand elements of arrays or objects.

- Copying arrays/objects:

```
const array = [1, 2, 3];  
const newArray = [...array]; // [1, 2, 3]
```

- Merging arrays/objects:

```
const arr1 = [1, 2];  
const arr2 = [3, 4];  
const mergedArray = [...arr1, ...arr2]; // [1, 2, 3, 4]
```

- Passing function arguments:

```
const nums = [1, 2, 3];  
console.log(Math.max(...nums)); // 3
```

Rest syntax (...)

The rest operator collects multiple elements into an array or object.

- Function parameters:

```
function sum(...numbers) {  
  return numbers.reduce((a, b) => a + b);  
}  
sum(1, 2, 3); // 6
```

- Destructuring:

```
const [first, ...rest] = [1, 2, 3];  
console.log(rest); // [2, 3]
```

[Explore spread and rest syntax on GreatFrontEnd →](#)

39. What are the differences between Maps vs. Plain objects?

Map

- Keys can be any type.
- Maintains the insertion order.
- Has a `size` property.
- Directly iterable.

```
const map = new Map();
map.set('key', 'value');
console.log(map.size); // 1
```

Plain objects

- Keys are strings or symbols.
- Iteration requires `Object.keys()` , `Object.values()` , or `Object.entries()` .
- No direct `size` property.

```
const obj = { key: 'value' };
console.log(Object.keys(obj).length); // 1
```

[Explore the difference between Map and plain objects on GreatFrontEnd →](#)

40. What are the differences between Map / Set and WeakMap / WeakSet

- **Key Types:** `WeakMap` and `WeakSet` keys must be objects, while `Map` and `Set` accept any data type.
- **Memory Management:** `WeakMap` and `WeakSet` allow garbage collection of keys, making them useful for managing memory.

- **Size:** Only Map and Set have a size property.
- **Iteration:** WeakMap and WeakSet are not iterable.

```
1 // Map Example
2 const map = new Map();
3 map.set({}, 'value');
4 console.log(map.size); // 1
5
6 // WeakMap Example
7 const weakMap = new WeakMap();
8 let obj = {};
9 weakMap.set(obj, 'value');
10 obj = null; // Key is garbage-collected
```

[Explore the differences between Map/Set and WeakMap/WeakSet on GreatFrontEnd →](#)

41. What are practical use cases for arrow functions?

Arrow functions simplify function syntax, making them ideal for inline callbacks.

```
1 // Traditional function syntax
2 const numbers = [1, 2, 3, 4, 5];
3 const doubledNumbers = numbers.map(function (number) {
4   return number * 2;
5 });
6 console.log(doubledNumbers); // [2, 4, 6, 8, 10]
7
8 // Arrow function syntax
9 const doubledWithArrow = numbers.map((number) => number * 2);
10 console.log(doubledWithArrow); // [2, 4, 6, 8, 10]
```

[Explore a use case for the new arrow function syntax on GreatFrontEnd →](#)

42. What are callback functions in asynchronous operations?

A callback is a function passed as an argument to another function, executed after the completion of an asynchronous task.

```
1  function fetchData(callback) {  
2      setTimeout(() => {  
3          const data = { name: 'John', age: 30 };  
4          callback(data);  
5      }, 1000);  
6  }  
7  
8  fetchData((data) => {  
9      console.log(data); // { name: 'John', age: 30 }  
10 });
```

[Explore the concept of a callback function in asynchronous operations on GreatFrontEnd →](#)

43. What is debouncing and throttling?

Debouncing delays execution of a function until a specified time has elapsed since its last invocation.

```
1  function debounce(func, delay) {  
2      let timeoutId;  
3      return (...args) => {  
4          clearTimeout(timeoutId);  
5          timeoutId = setTimeout(() => func.apply(this, args), delay);  
6      };  
7  }
```

Throttling ensures a function executes at most once within a set time interval.

```
1  function throttle(func, limit) {  
2    let inThrottle;  
3    return (...args) => {  
4      if (!inThrottle) {  
5        func.apply(this, args);  
6        inThrottle = true;  
7        setTimeout(() => (inThrottle = false), limit);  
8      }  
9    };  
10 }
```

[Explore the concept of debouncing and throttling on GreatFrontEnd →](#)

44. How does destructuring assignment work?

Destructuring simplifies extracting values from arrays or objects into individual variables.

```
1  // Array destructuring  
2  const [a, b] = [1, 2];  
3  
4  // Object destructuring  
5  const { name, age } = { name: 'John', age: 30 };
```

[Explore the concept of destructuring assignment on GreatFrontEnd →](#)

45. What is function hoisting?

Hoisting moves function declarations to the top of their scope during the compilation phase. However, function expressions and arrow functions do not get hoisted in the same way.

```
1  // Function declaration  
2  hoistedFunction(); // Works fine
```

```
3  function hoistedFunction() {
4    console.log('This function is hoisted');
5  }
6
7  // Function expression
8  nonHoistedFunction(); // Throws an error
9  var nonHoistedFunction = function () {
10   console.log('This function is not hoisted');
11 };
```

[Explore the concept of hoisting on GreatFrontEnd →](#)

46. How does inheritance work in ES2015 classes?

Classes in ES2015 use `extends` for inheritance and `super` to access parent constructors and methods.

```
1  class Animal {
2    constructor(name) {
3      this.name = name;
4    }
5
6    speak() {
7      console.log(`${this.name} makes a noise.`);
8    }
9  }
10
11 class Dog extends Animal {
12   constructor(name, breed) {
13     super(name);
14     this.breed = breed;
15   }
16
17   speak() {
18     console.log(`${this.name} barks.`);
19   }
20 }
21
```

```
22  const dog = new Dog('Rex', 'German Shepherd');
23  dog.speak(); // Rex barks.
```

[Explore the concept of inheritance in ES2015 classes on GreatFrontEnd →](#)

47. What is lexical scoping?

Lexical scoping determines variable access based on where functions are defined, not where they're called.

```
1  function outerFunction() {
2    let outerVariable = 'I am outside!';
3
4    function innerFunction() {
5      console.log(outerVariable); // I am outside!
6    }
7
8    innerFunction();
9  }
10
11  outerFunction();
```

[Explore the concept of lexical scoping on GreatFrontEnd →](#)

48. What are scopes in JavaScript?

JavaScript has three main types of scope: global, function, and block.

```
1  // Global scope
2  var globalVar = 'I am global';
3
4  function myFunction() {
5    // Function scope
```

```
6   var functionVar = 'I am in a function';
7
8   if (true) {
9     // Block scope
10    let blockVar = 'I am in a block';
11    console.log(blockVar); // Accessible here
12  }
13  // console.log(blockVar); // Error
14 }
```

[Explore the concept of scope in JavaScript on GreatFrontEnd →](#)

49. What is the spread operator?

The spread operator (`...`) expands elements of an iterable (like arrays) or properties of objects into individual elements.

```
1  // Copying an array
2  const arr1 = [1, 2, 3];
3  const arr2 = [...arr1];
4
5  // Merging arrays
6  const mergedArray = [...arr1, [4, 5]];
7
8  // Copying an object
9  const obj1 = { a: 1, b: 2 };
10 const obj2 = { ...obj1 };
11
12 // Passing as function arguments
13 const sum = (x, y, z) => x + y + z;
14 const nums = [1, 2, 3];
15 sum(...nums); // 6
```

[Explore the spread operator on GreatFrontEnd →](#)

50. How does `this` work in event handlers?

In JavaScript, `this` in event handlers refers to the element that triggered the event. Its context can be explicitly bound using `bind()`, arrow functions, or direct assignment.

```
1  const button = document.querySelector('button');
2  button.addEventListener('click', function () {
3    console.log(this); // Refers to the button
4  });
5
6  const obj = {
7    handleClick: function () {
8      console.log(this); // Refers to obj
9    },
10 };
11 button.addEventListener('click', obj.handleClick.bind(obj));
```

[Explore the concept of `this` in event handlers on GreatFrontEnd →](#)

Modern JavaScript (ES2020+)

The next 25 questions cover ES2020–ES2025 additions that come up in modern JavaScript interviews.

51. What does optional chaining (`?.`) do, and where does it short-circuit?

Optional chaining (`?.`) returns `undefined` if the value to its left is `null` or `undefined`, instead of throwing. It works on property access, function calls, and array indexing.

```
1  const user = { profile: null };
2
3  console.log(user.profile?.name); // undefined – no throw
```

```

4 console.log(user.callbacks?.onSave?.()); // undefined
5 console.log(user.tags?.[0]); // undefined

```

Important: it short-circuits the **rest of the chain** the moment any `?.` operand is nullish, so `user?.a.b.c` will short-circuit at `user?` and never evaluate `.a.b.c`. It does **not** protect against `0`, `''`, `false`, or `NaN` — those are not nullish.

52. What is nullish coalescing (`??`) and how does it differ from `||` ?

`??` returns its right-hand side only when the left side is `null` or `undefined`. `||` returns the right-hand side for any falsy value — `0`, `''`, `false`, `NaN`, `null`, `undefined`.

```

1 const port = 0;
2
3 console.log(port || 3000); // 3000 - 0 is falsy
4 console.log(port ?? 3000); // 0 - 0 is not nullish
5
6 const name = '';
7 console.log(name || 'Anonymous'); // 'Anonymous'
8 console.log(name ?? 'Anonymous'); // ''

```

Use `??` when "no value provided" is the only case you want to replace. `||` is right when any falsy value should be treated as "missing" — like defaulting a flag.

53. What are logical assignment operators (`||=` , `&&=` , `??=`)?

ES2021's logical assignment operators combine a logical check with assignment, assigning only when the check passes.

```

1  const config = { retries: 0, host: '' };
2
3  config.host ||= 'localhost'; // assigns - '' is falsy
4  config.retries ??= 3; // does NOT assign - 0 is not nullish
5  config.debug &&= 'verbose'; // does NOT assign - debug is undefined

```

- `a ||= b` → assigns `b` only if `a` is falsy.
- `a &&= b` → assigns `b` only if `a` is truthy.
- `a ??= b` → assigns `b` only if `a` is nullish.

They're short-circuiting: the right side runs only when the assignment will happen.

54. What does this print, and why? (`var` vs `let` in a `setTimeout` loop)

```

for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0);
}
// 3, 3, 3

```

`var` is function-scoped, so all three callbacks close over the same `i`. By the time the timers fire (after the synchronous loop finishes), `i` is `3`.

Swap `var` for `let`:

```

for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0);
}
// 0, 1, 2

```

`let` creates a fresh binding per iteration, so each callback captures its own `i`.

Pre- `let`, the workaround was an IIFE:

```
1  for (var i = 0; i < 3; i++) {  
2    (function (j) {  
3      setTimeout(() => console.log(j), 0);  
4    })(i);  
5  }
```

55. How does `Promise.allSettled()` differ from `Promise.all()` ?

`Promise.all()` rejects as soon as any input promise rejects, and you lose the results of the others. `Promise.allSettled()` waits for every promise to settle and returns an array describing each outcome — `{ status: 'fulfilled', value }` or `{ status: 'rejected', reason }`.

```
1  const results = await Promise.allSettled([  
2    fetch('/api/a'),  
3    fetch('/api/b'),  
4    fetch('/api/c'),  
5  ]);  
6  
7  for (const result of results) {  
8    if (result.status === 'fulfilled') {  
9      console.log('ok:', result.value);  
10   } else {  
11     console.error('failed:', result.reason);  
12   }  
13 }
```

Use `allSettled` when you want every result regardless of failures — e.g., loading multiple independent widgets where one failure shouldn't blank the page.

[Explore the difference between Promise.all and Promise.allSettled on GreatFrontEnd →](#)

56. What is Promise.any() and how does it handle rejections?

Promise.any() resolves with the value of the first promise to fulfill. It only rejects if **all** input promises reject, and the rejection is an AggregateError containing every individual reason.

```
1  try {
2    const fastest = await Promise.any([
3      fetch('https://mirror-1.example.com/data'),
4      fetch('https://mirror-2.example.com/data'),
5      fetch('https://mirror-3.example.com/data'),
6    ]);
7    console.log('first to respond:', fastest);
8  } catch (err) {
9    console.error(err.errors); // array of all rejection reasons
10 }
```

Use it for racing mirrors, fallbacks, or "first response wins" patterns.

57. How do you cancel a fetch request with AbortController ?

AbortController exposes a signal you pass to any abort-aware API (fetch , addEventListener , streams, observers). Calling .abort() cancels the operation and rejects the associated promise with an AbortError .

```
1  const controller = new AbortController();
2
3  const promise = fetch('/api/slow', { signal: controller.signal });
4
5  // Cancel after 5 seconds
```

```
6   setTimeout(() => controller.abort(), 5000);
7
8   try {
9     const res = await promise;
10    console.log(await res.json());
11  } catch (err) {
12    if (err.name === 'AbortError') {
13      console.log('Request cancelled');
14    }
15  }
```

Common uses: cancelling in-flight requests when a component unmounts, debouncing search-as-you-type, and timing out long requests. You can also pass the same signal to multiple operations to cancel them as a group.

[Explore aborting web requests with AbortController on GreatFrontEnd →](#)

58. What's the difference between `async/await` and raw Promises?

`async/await` is syntactic sugar over Promises. Both run the same machinery; the difference is readability and control flow.

```
1  // Raw promise chain
2  function loadUser(id) {
3    return fetch(`/users/${id}`)
4      .then((res) => res.json())
5      .then((user) => fetch(`/orgs/${user.orgId}`))
6      .then((res) => res.json());
7  }
8
9  // async/await equivalent
10 async function loadUser(id) {
11   const user = await (await fetch(`/users/${id}`)).json();
12   const org = await (await fetch(`/orgs/${user.orgId}`)).json();
13   return org;
14 }
```

```
14 }
```

`async/await` lets you use `try/catch`, regular `if / for`, and reads top-to-bottom. Raw Promises shine when you want parallelism (`Promise.all`) or compose pipelines functionally.

[Explore how `async/await` simplifies asynchronous code on GreatFrontEnd →](#)

59. What are generators, and how is `async/await` related to them?

A generator (`function*`) is a function that can pause and resume. Each `yield` suspends execution and returns a value to the caller; `next()` resumes it.

```
1  function* counter() {  
2    yield 1;  
3    yield 2;  
4    yield 3;  
5  }  
6  
7  const c = counter();  
8  console.log(c.next()); // { value: 1, done: false }  
9  console.log(c.next()); // { value: 2, done: false }  
10 console.log(c.next()); // { value: 3, done: true }
```

You can also send values **back in** via `next(value)`, making generators bidirectional coroutines:

```
1  function* dialog() {  
2    const name = yield 'What is your name?';  
3    yield `Hello, ${name}!`;  
4  }  
5  
6  const d = dialog();  
7  d.next(); // { value: 'What is your name?', done: false }
```

```
8 d.next('Ada'); // { value: 'Hello, Ada!', done: false }
```

`async / await` is sugar over generators — an `async` function is a generator that yields promises, with the runtime calling `.next()` when each resolves. `redux-saga` uses generators directly to make `async` control flow testable.

60. What is `error.cause` and why is it useful?

`error.cause` (ES2022) is a standard way to attach the underlying error when re-throwing — preserving the original error and its stack without string-mashing.

```
1  async function loadUser(id) {
2    try {
3      return await fetchUser(id);
4    } catch (originalError) {
5      throw new Error(`Failed to load user ${id}`, { cause: originalError });
6    }
7  }
8
9  try {
10   await loadUser(42);
11 } catch (err) {
12   console.error(err.message); // 'Failed to load user 42'
13   console.error(err.cause); // the original network error, with full stack
14 }
```

Before `cause`, the inner error was usually stringified into the outer message (`new Error('Failed: ' + e.message)`), losing the inner stack. With `cause`, devtools, `console.error`, and logging libraries (Sentry, Pino) walk the chain automatically.

Use it at every boundary where you wrap a low-level error into a higher-level one.

61. What are the immutable array methods (`toSorted` , `toReversed` , `toSpliced` , `with`)?

ES2023 added four methods that return a **new** array instead of mutating the original. They mirror their classic counterparts but are safe with frozen data, React state, or any place where mutation causes bugs.

```
1  const arr = [3, 1, 2];
2
3  const sorted = arr.toSorted(); // [1, 2, 3] – arr unchanged
4  const reversed = arr.toReversed(); // [2, 1, 3] – arr unchanged
5  const spliced = arr.toSpliced(1, 1, 9, 9); // [3, 9, 9, 2]
6  const replaced = arr.with(0, 99); // [99, 1, 2]
7
8  console.log(arr); // [3, 1, 2] – still original
```

Before these methods, you had to write `[...arr].sort()` or `arr.slice().reverse()` to avoid mutating. They make immutable-style code one call shorter and clearer.

62. What does `Array.prototype.findLast()` do?

`findLast()` (ES2023) returns the **last** element that matches a predicate. `findLastIndex()` returns its index. They're the mirror of `find()` / `findIndex()`.

```
1  const events = [
2    { type: 'click', t: 100 },
3    { type: 'scroll', t: 200 },
4    { type: 'click', t: 300 },
5  ];
6
7  const lastClick = events.findLast((e) => e.type === 'click');
8  console.log(lastClick); // { type: 'click', t: 300 }
```

The pre-2023 alternative was `[...arr].reverse().find(...)` — $O(n)$ extra work and harder to read.

63. What is `Object.groupBy()` / `Map.groupBy()` ?

`Object.groupBy()` and `Map.groupBy()` (ES2024) group an iterable's items by the return value of a callback. `Object.groupBy` uses string keys; `Map.groupBy` allows any value as a key.

```
1  const people = [  
2    { name: 'Ada', team: 'eng' },  
3    { name: 'Lin', team: 'design' },  
4    { name: 'Rao', team: 'eng' },  
5  ];  
6  
7  const byTeam = Object.groupBy(people, (p) => p.team);  
8  // { eng: [{Ada}, {Rao}], design: [{Lin}] }  
9  
10 const teamObj = { id: 1 };  
11 const byRef = Map.groupBy(people, (p) => (p.team === 'eng' ? teamObj : null));  
12 // Map { teamObj => [{Ada}, {Rao}], null => [{Lin}] }
```

They replace the `reduce((acc, x) => ...)` grouping pattern.

64. What are the new Set methods (union , intersection , difference)?

ES2025 added set-algebra methods to `Set`. They take **any iterable** on the right side (not just another `Set`) and always return a new `Set`.

```
1  const a = new Set([1, 2, 3]);  
2  const b = new Set([3, 4, 5]);  
3  
4  a.union(b); // Set { 1, 2, 3, 4, 5 }
```

```
5  a.intersection(b); // Set { 3 }
6  a.difference(b); // Set { 1, 2 }
7  a.symmetricDifference(b); // Set { 1, 2, 4, 5 }
8
9  a.isSubsetOf(b); // false
10 a.isSupersetOf(b); // false
11 a.isDisjointFrom(b); // false
```

Before this, an intersection was `new Set([...a].filter(x => b.has(x)))`. The native methods are also faster — implementations iterate the smaller side.

65. What are iterator helpers (`.map` , `.filter` , `.take` on iterators)?

ES2025 iterator helpers add `.map` , `.filter` , `.take` , `.drop` , `.flatMap` , `.reduce` , `.some` , `.every` , `.find` , and `.toArray` directly to iterators (and generators). Unlike array methods, they're **lazy** — values flow through one at a time.

```
1  function* naturals() {
2    let n = 1;
3    while (true) yield n++;
4  }
5
6  const firstFiveSquares = naturals()
7    .map((n) => n * n)
8    .take(5)
9    .toArray();
10
11 console.log(firstFiveSquares); // [1, 4, 9, 16, 25]
```

Laziness means infinite sequences don't allocate everything, and pipelines don't materialize intermediate arrays.

`AsyncIterator.prototype.map` and friends do the same over `for await...of` sources.

66. What is `Array.fromAsync` ?

`Array.fromAsync` (ES2024) is the async counterpart of `Array.from`. It awaits each value from an async iterable (or a sync iterable of promises) and collects them into an array.

```
1  async function* fetchPages(urls) {
2    for (const url of urls) {
3      const res = await fetch(url);
4      yield await res.json();
5    }
6  }
7
8  const allPages = await Array.fromAsync(
9    fetchPages(['/api/p/1', '/api/p/2', '/api/p/3']),
10  );
```

Pages are fetched **sequentially**. For parallel fetching, use `Promise.all(urls.map(fetch))` instead. `Array.fromAsync` fits when each step depends on the previous, or when you just want to materialize an async iterable.

67. How do you make a custom iterable with `Symbol.iterator` ?

Any object with a `[Symbol.iterator]()` method becomes iterable — usable with `for...of`, `spread`, `destructuring`, and `Array.from`. The method must return an iterator (an object with `next()` returning `{ value, done }`).

The easy way is to delegate to a generator:

```
1  class Range {
2    constructor(from, to) {
3      this.from = from;
4      this.to = to;
5    }
6  }
```

```

7      *[Symbol.iterator]() {
8          for (let n = this.from; n <= this.to; n++) yield n;
9      }
10 }
11
12 const r = new Range(1, 4);
13
14 for (const n of r) console.log(n); // 1 2 3 4
15 console.log([...r]); // [1, 2, 3, 4]
16 const [first, ...rest] = r; // 1, [2, 3, 4]

```

For async sources, implement `[Symbol.asyncIterator]()` instead and consume with `for await...of`. `NodeList`, `Map`, `Set`, and `Node` streams all expose themselves through this same protocol.

68. What's the difference between shallow and deep copy?

A **shallow copy** duplicates the top level only — nested objects are shared by reference. A **deep copy** recursively duplicates everything.

```

1  const original = { user: { name: 'Ada' }, tags: ['x'] };
2
3  // Shallow copies – nested refs shared
4  const a = { ...original };
5  const b = Object.assign({}, original);
6  a.user.name = 'Lin';
7  console.log(original.user.name); // 'Lin' – mutation leaked
8
9  // Deep copies
10 const c = JSON.parse(JSON.stringify(original)); // lossy
11 const d = structuredClone(original); // preferred

```

Pick by use case:

- `{...obj}` / `Object.assign` : fast, fine when you only mutate the top level.

- `JSON.parse(JSON.stringify(...))` : deep but lossy — `undefined` , `functions` , `Date` , `Map` , `Set` , `RegExp` , cycles all break. Avoid in 2026.
- `structuredClone` : deep, preserves `Date` / `Map` / `Set` and cycles. Default deep-copy choice.
- **Library** `cloneDeep` (**lodash**): handles things `structuredClone` doesn't (functions, prototypes) — at the cost of bundle size.

The classic React bug — `setState({ ...state, list: state.list })` followed by mutating `state.list` — is shallow-copy leakage.

69. What is `structuredClone()` and how is it different from `JSON.parse(JSON.stringify(...))` ?

`structuredClone()` is a built-in that deep-clones a value using the structured clone algorithm. Unlike the `JSON` round-trip, it handles `Date` , `Map` , `Set` , `RegExp` , `ArrayBuffer` , typed arrays, and cyclic references correctly.

```
1  const original = {
2    date: new Date(),
3    map: new Map([[ 'k', 'v' ]]),
4    set: new Set([1, 2, 3]),
5  };
6  original.self = original; // cycle
7
8  const copy = structuredClone(original);
9
10 console.log(copy.date instanceof Date); // true
11 console.log(copy.map instanceof Map); // true
12 console.log(copy.self === copy); // true – cycle preserved
```

It does **not** clone functions, DOM nodes, or prototypes. For plain-data deep copies, prefer `structuredClone` over the `JSON` trick — it's safer and faster.

70. What are private class fields (#field)?

Private fields, prefixed with `#`, are accessible only inside the class that declares them. They're enforced by the language — not a naming convention like `_field`.

```
1  class Counter {
2    #count = 0;
3
4    increment() {
5      this.#count++;
6    }
7
8    get value() {
9      return this.#count;
10   }
11 }
12
13 const c = new Counter();
14 c.increment();
15 console.log(c.value); // 1
16 console.log(c.#count); // SyntaxError
```

Subclasses can't reach private fields of their parents, and `Object.keys()` / `for...in` won't enumerate them. Use them when you need true encapsulation — internal state that consumers (or future subclasses) must not touch.

71. ES Modules vs CommonJS — what's the difference?

CommonJS (`require` / `module.exports`) is Node.js's legacy module system. ES Modules (`import` / `export`) are the standard, supported in browsers and modern Node.js.

```
1  // CommonJS
2  const fs = require('fs');
3  module.exports = { foo: 1 };
4
5  // ES Modules
```

```
6  import fs from 'node:fs';
7  export const foo = 1;
```

Key differences:

- **Loading:** CommonJS is synchronous and runtime-resolved. ES Modules are statically analyzed, asynchronous, and support top-level `await`.
- **Bindings:** CommonJS exports are values (copies). ES Module exports are live bindings — importers see updates.
- **Tree-shaking:** ESM's static structure lets bundlers eliminate unused exports; CommonJS generally can't.
- **Interop:** Node lets ESM `import` from CommonJS, but a CommonJS `require` of ESM needs `await import()`.

New code should default to ESM. Use CommonJS only when targeting old Node or legacy tooling.

[Explore the differences between CommonJS and ES Modules on GreatFrontEnd →](#)

72. What is dynamic `import()` and when would you use it?

Dynamic `import()` is a function-like syntax that loads a module at runtime and returns a

Flash Sale: 30% off everything with code FLASH30 →

×



Get started Prepare ▾

Get full access



```
button.addEventListener('click', async () => {
  const { default: Chart } = await import('./Chart.js');
  new Chart(document.getElementById('canvas')).render();
});
```

Common uses:

- **Code splitting:** defer loading large dependencies until needed (charts, editors, modals).
- **Conditional loading:** pick between modules based on locale, feature flag, or platform.
- **Polyfills:** load only when the target lacks a feature.

Most bundlers (webpack, Vite, esbuild) treat dynamic `import()` as a code-split boundary automatically.

73. What are tagged template literals?

A tagged template literal lets a function process a template string. The tag function receives the static string parts as its first argument and the interpolated values as the rest.

```

1  function html(strings, ...values) {
2    return strings.reduce((out, str, i) => {
3      const safe = values[i] != null ? escapeHtml(values[i]) : '';
4      return out + str + safe;
5    }, '');
6  }
7
8  const name = '<script>alert(1)</script>';
9  const out = html`<p>Hello, ${name}!</p>`;
10 // "<p>Hello, &lt;script&gt;alert(1)&lt;/script&gt;!</p>"

```

Use them for safe HTML/SQL building, i18n message formatting, GraphQL queries (`gql`...``), or styled-components-style CSS-in-JS.

[Explore tagged templates on GreatFrontEnd →](#)

74. What does `String.prototype.replaceAll()` do?

`replaceAll()` (ES2021) replaces **every** occurrence of a substring or pattern. Before it, `replace` with a string argument only replaced the first match, forcing a global regex (`/.../g`)

for the common case.

```
1  const s = 'cat, cat, cat';
2
3  console.log(s.replace('cat', 'dog')); // 'dog, cat, cat'
4  console.log(s.replaceAll('cat', 'dog')); // 'dog, dog, dog'
5  console.log(s.replaceAll(/CAT/gi, 'dog')); // 'dog, dog, dog'
```

If you pass a regex, it **must** have the `g` flag — otherwise `replaceAll` throws. The function form of the second argument is supported, same as `replace`.

75. What is `BigInt` and when should you use it?

`BigInt` is a primitive type for integers of arbitrary size, written with an `n` suffix or via `BigInt(...)`. Standard `number` loses precision beyond $2^{53} - 1$ (`Number.MAX_SAFE_INTEGER`); `BigInt` does not.

```
const big = 9007199254740993n;
console.log(big + 1n); // 9007199254740994n

console.log(Number.MAX_SAFE_INTEGER + 2); // 9007199254740992 — wrong
```

Caveats: you can't mix `BigInt` and `Number` in arithmetic (`1n + 1` throws), `Math.*` doesn't accept `BigInt`, and `JSON.stringify` throws on `BigInt`. Reach for it when handling 64-bit IDs, monetary values in minor units, cryptography, or timestamps in nanoseconds — anywhere 2^{53} is a real ceiling.

Conclusion

That's the list. The next step is practice — work through each question until you can explain the concept and write the code from scratch without looking.

More JavaScript interview prep:

- [Front End Interview Playbook](#)
- [Practice JavaScript coding interview questions](#)
- [Practice JavaScript quiz interview questions](#)

A broader set of [+190 JavaScript interview questions](#) is also available in our GitHub repo.

[Copy link](#)[Share](#)[← Previous](#)[Next →](#)

30 Basic to Advanced React Interview Questions with Solutions

100+ React Interview Questions Straight from Ex-interviewers (2026)



Subscribe to our newsletter

[Notify me](#)

Practice

Get started

JavaScript functions

User interface coding

System design

Quiz

Study Plans

1 Week Plan

1 Month Plan

3 Months Plan

Company

Pricing

Guides

[Front End Interview Playbook](#)

[Front End System Design Playbook](#)

[React Interview Playbook](#)

[Behavioral Interview Playbook](#)

Promotions

[Roadmap](#)

[About](#)

[Team](#)

[Contact us](#)

[Advertise with us](#) New

[Become an affiliate](#)

[Careers](#)

[Blog](#)

[Medium](#)

[DEV Community](#)



English (US) ▾

© 2026 Codeney Pte Ltd. All rights reserved.

[Privacy Policy](#)

[Terms of Service](#)